May 2026

UNDERSTANDING CONTEXT IN GO

By Jerry Agbesi

CONTENT

THE PROBLEM

- In a Go server, each request is handled by its own goroutine
- Hence, there needs to be a way to manage metadata on individual requests
- Metadata could be in two categories:
  - Metadata required to process a request correctly.
  - Metadata on when to stop processing.
- In other languages, eg, Java, **threadlocal** variables are used to solve this.
- Go can’t do this because goroutines don’t have unique identities that can be used to look up values.
- What does Go do instead? Carry the metadata explicitly along with the request.
- ….Introducing the **Context**.

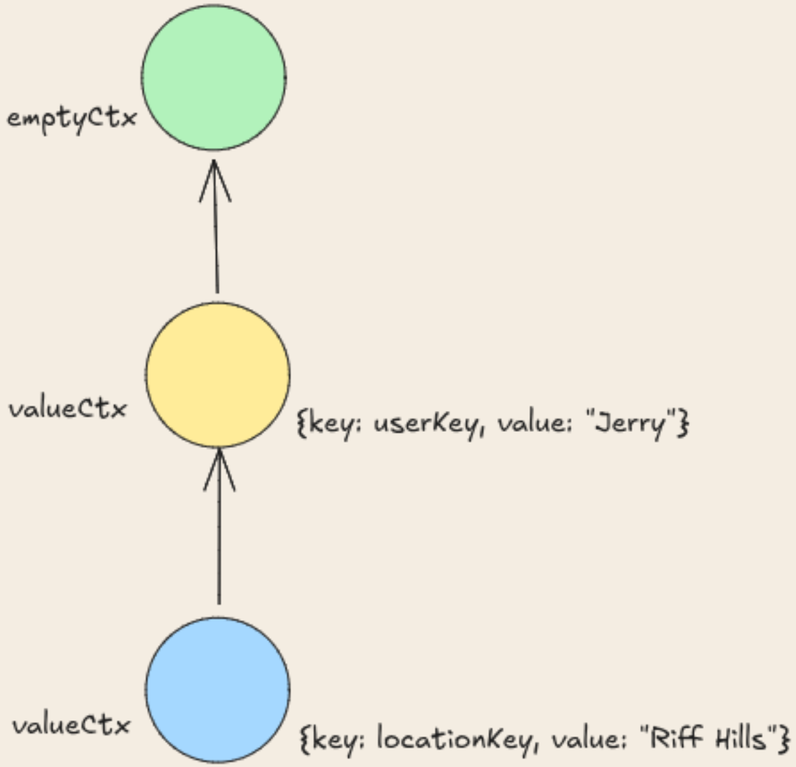
WHAT'S THE CONTEXT?

- A context is simply an instance that meets the Context interface defined in the context package.
- It's another parameter to your function, and by convention, the context is passed through your program as the first parameter of a function
- A context can be used to pass data, such as authentication data, Tenant ID, locale/language and logger instance to functions or goroutines that need it downstream.

```
type Context interface {

    Deadline() (deadline time.Time, ok bool)

    Done() <-chan struct{}

    Err() error

    Value(key any) any
}
```

THE CONTEXT TREE

- A context is treated as an immutable instance.
- An empty context is a starting point. Each time you add some metadata you do so by **wrapping** the existing parent context with a child context.
- **NB:** The context is never used to pass information out of deeper layers to higher layers.



VALUES

- Passing explicit data through functions in Go is always preferred.
- But in situations where this won’t be so efficient, e.g., if you want to make a value available to your handler in a middleware, you need to store it in the context.
- A factory method **context.WithValue** provides a way to associate request-scoped values with a Context.
- **ctx.Value(key)** walks up the context tree to find the value — it checks the current context and all its parents.

```go
package main

import (
    "context"
    "fmt"
)

type ctxKey int

const (
    _ ctxKey = iota
    userKey
    locationKey
)

func main() {
    ctx := context.Background()

    ctx = context.WithValue(ctx, userKey, "Jerry")
    ctx = context.WithValue(ctx, locationKey, "Riff Hills")

    user := ctx.Value(userKey).(string)
    location := ctx.Value(locationKey).(string)

    fmt.Println("User: ", user, "Location: ", location)
}
```

CANCELLATION

- In a long-running operation with multiple goroutines, you need a way to signal all of them to stop.
- This is handled by creating a cancellable context
- There are 2 mechanisms for creating a cancellable context:
    - **WithCancel** gives you a context and a function, when you call that function, every goroutine watching the context knows to stop.
    - **WithCancelCause** works the same way, but when you call the cancel function you can pass in an error, and any part of your program that has access to the context can ask why it was cancelled.

CONTEXTS WITH DEADLINES

- A server is a shared resource and has the responsibility to manage itself to provide a **fair** amount of time to all its users.
- The context provides a way to control how long a request runs.
- There are two main functions used to create a time-limited context
    - **context.WithTimeout**: takes in an existing context and **time.Duration** that specifies the duration until the context automatically cancels.
    - **context.WithDeadline**: takes in an existing context and a **time.Time** that specifies the time when the context is automatically canceled.
- Any timeout that you set on the child context is bounded by the timeout set on the parent context.
- **NB:** If you pass a time in the past to **context.WithDeadline**, the context is created already canceled.

GOLDEN RULES

- Always pass the context as the first argument to functions
- Always defer cancel() to prevent leaks
- The context with timeout should usually be the parent in a tree of cancellable contexts.
- Let the libraries (eg. database/sql) do the heavy lifting for cancellation

May 2026

THANK YOU

Any Questions?